Zellic

April 3, 2026

Prepared for
Javed Khan
Celestia Labs

Prepared by
Jinseo Kim
Quentin Lemauf
Moritz Schneider
Leonardo Veronese
Zellic

nitro-das-celestia

Smart Contract and Web Security Assessment

Contents

About Zellic **4**

1. Overview **4**

1.1. Executive Summary 5

1.2. Goals of the Assessment 5

1.3. Non-goals and Limitations 5

1.4. Results 5

2. Introduction **6**

2.1. About nitro-das-celestia 7

2.2. Methodology 7

2.3. Scope 9

2.4. Project Overview 9

2.5. Project Timeline 10

3. Detailed Findings **10**

3.1. Continuation read proof does not check the real payload size 11

3.2. Read preimage proof does not bind proved shares to the share picked by the certificate 13

3.3. Validating the certificate depends on `BlobstreamX.verifyAttestation` terminating 16

3.4. The function `validateCertificate` may return false for a valid proof under some gas limits 18

3.5. Unsafe `uint64` to `big.Int` conversion 20

3.6. Missing retry logic on postsubmission read operations 22

3.7. Possible infinite loop in backwards search in `filter()` 24

3.8. Noncompliant error messages for invalid certificate errors in reader methods 25

3.9. Noncompliant error in the case of an empty batch 27

3.10. Insufficient test coverage 28

4. Discussion 29

4.1. Potential memory-corruption risk in prefix slice reuse 30

4.2. Unused `timeout` parameter in `writerForCelestia.Store` RPC method 31

4.3. Silent validation failure in `GetProof` 31

4.4. Iterator error handling causes silent discard of a valid event in `filter` 32

5. System Design 33

5.1. Component: DA server 34

5.2. Component: DA proof validator smart contract 37

6. Assessment Results 39

6.1. Disclaimer 40

About Zellic

Zellic is a vulnerability research firm with deep expertise in blockchain security. We specialize in EVM, Move (Aptos and Sui), and Solana as well as Cairo, NEAR, and Cosmos. We review L1s and L2s, cross-chain protocols, wallets and applied cryptography, zero-knowledge circuits, web applications, and more.

Prior to Zellic, we founded the #1 CTF (competitive hacking) team ↗ worldwide in 2020, 2021, and 2023. Our engineers bring a rich set of skills and backgrounds, including cryptography, web security, mobile security, low-level exploitation, and finance. Our background in traditional information security and competitive hacking has enabled us to consistently discover hidden vulnerabilities and develop novel security research, earning us the reputation as the go-to security firm for teams whose rate of innovation outpaces the existing security landscape.

For more on Zellic's ongoing security research initiatives, check out our website zellic.io ↗ and follow @zellic_io ↗ on Twitter. If you are interested in partnering with Zellic, contact us at hello@zellic.io ↗.

1. Overview

1.1. Executive Summary

Zellic conducted a security assessment for Celestia Labs from March 20th to March 24th, 2026. During this engagement, Zellic reviewed nitro-das-celestia's code for security vulnerabilities, design issues, and general weaknesses in security posture.

1.2. Goals of the Assessment

In a security assessment, goals are framed in terms of questions that we wish to answer. These questions are agreed upon through close communication between Zellic and the client. In this assessment, we sought to answer the following questions:

- Are there Solidity bugs in the on-chain validator smart contract? Can a malicious validator abuse the challenge mechanism to create a malicious fork?
- Does the project adhere to the specifications and requirements of the Arbitrum Nitro stack for the integration with the DA API?
- Does the `validateCertificate` function terminate correctly without reverting?

1.3. Non-goals and Limitations

We did not assess the following areas that were outside the scope of this engagement:

- All autogenerated files, test and mocks
- The Celestia node software
- The Arbitrum Nitro components that connect to the DA server and contract
- Third-party dependencies and libraries
- Front-end components
- Infrastructure relating to the project
- Key custody

Due to the time-boxed nature of security assessments in general, there are limitations in the coverage an assessment can provide.

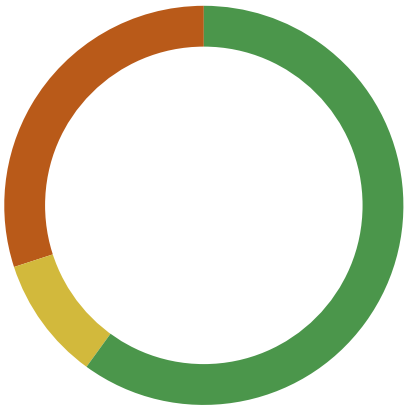
1.4. Results

During our assessment on the scoped nitro-das-celestia contracts, we discovered 10 findings. No critical issues were found. Three findings were of high impact, one was of medium impact, and six were of low impact.

Additionally, Zellic recorded its notes and observations from the assessment for the benefit of Celestia Labs in the Discussion section (4. ↗).

Breakdown of Finding Impacts

Impact Level	Count
Critical	0
High	3
Medium	1
Low	6
Informational	0



2. Introduction

2.1. About nitro-das-celestia

Celestia Labs contributed the following description of nitro-das-celestia:

> nitro-das-celestia implements Celestia backed data availability for Arbitrum Nitro.

2.2. Methodology

During a security assessment, Zellic works through standard phases of security auditing, including both automated testing and manual review. These processes can vary significantly per engagement, but the majority of the time is spent on a thorough manual review of the entire scope.

Alongside a variety of tools and analyzers used on an as-needed basis, Zellic focuses primarily on the following classes of security and reliability issues:

Basic coding mistakes. Many critical vulnerabilities in the past have been caused by simple, surface-level mistakes that could have easily been caught ahead of time by code review. Depending on the engagement, we may also employ sophisticated analyzers such as model checkers, theorem provers, fuzzers, and so on as necessary. We also perform a cursory review of the code to familiarize ourselves with the contracts.

Business logic errors. Business logic is the heart of any smart contract application. We examine the specifications and designs for inconsistencies, flaws, and weaknesses that create opportunities for abuse. For example, these include problems like unrealistic tokenomics or dangerous arbitrage opportunities. To the best of our abilities, time permitting, we also review the contract logic to ensure that the code implements the expected functionality as specified in the platform's design documents.

Integration risks. Several well-known exploits have not been the result of any bug within the contract itself; rather, they are an unintended consequence of the contract's interaction with the broader DeFi ecosystem. Time permitting, we review external interactions and summarize the associated risks: for example, flash loan attacks, oracle price manipulation, MEV/sandwich attacks, and so on.

Code maturity. We look for potential improvements in the codebase in general. We look for violations of industry best practices and guidelines and code quality standards. We also provide suggestions for possible optimizations, such as gas optimization, upgradability weaknesses, centralization risks, and so on.

For each finding, Zellic assigns it an impact rating based on its severity and likelihood. There is no hard-and-fast formula for calculating a finding's impact. Instead, we assign it on a case-by-case basis based on our judgment and experience. Both the severity and likelihood of an issue affect its impact. For instance, a highly severe issue's impact may be attenuated by a low likelihood. We assign the following impact ratings (ordered by importance): Critical, High, Medium, Low, and

Informational.

Zellic organizes its reports such that the most important findings come first in the document, rather than being strictly ordered on impact alone. Thus, we may sometimes emphasize an "Informational" finding higher than a "Low" finding. The key distinction is that although certain findings may have the same impact rating, their *importance* may differ. This varies based on various soft factors, like our clients' threat models, their business needs, and so on. We aim to provide useful and actionable advice to our partners considering their long-term goals, rather than a simple list of security issues at present.

Finally, Zellic provides a list of miscellaneous observations that do not have security impact or are not directly related to the scoped contracts itself. These observations — found in the Discussion (4. ↗) section of the document — may include suggestions for improving the codebase, or general recommendations, but do not necessarily convey that we suggest a code change.

2.3. Scope

The engagement involved a review of the following targets:

nitro-das-celestia Contracts

Types	Solidity, Go
Platforms	EVM-compatible, Web
Target	nitro-das-celestia
Repository	https://github.com/celestiaorg/nitro-das-celestia ↗
Version	`0872b54ed1d93d8c866266a68111bbf53b97044b`

Programs	`contracts/src/**/*.sol`
	`daserver/**/*`
	`Excluding daserver/bridgegen/*`

2.4. Project Overview

Zellic was contracted to perform a security assessment for a total of 1.5 person-weeks. The assessment was conducted by four consultants over the course of three calendar days.

Contact Information

The following project manager was associated with the engagement:

Chad McDonald
Engagement Manager
chad@zellic.io

The following consultants were engaged to conduct the assessment:

Jinseo Kim
Engineer
jinseo@zellic.io

Quentin Lemauf
Engineer
quentin@zellic.io

Moritz Schneider
Engineer
moritz@zellic.io

Leonardo Veronese
Engineer
leonardo@zellic.io

2.5. Project Timeline

The key dates of the engagement are detailed below.

March 20, 2026	Start of primary review period
March 23, 2026	Kick-off call
March 24, 2026	End of primary review period

3. Detailed Findings

3.1. Continuation read proof does not check the real payload size

Target	CelestiaDAProofValidator.sol		
Category	Soundness	**Severity**	High
Likelihood	Medium	**Impact**	High

Description

The function `validateReadPreimage()` uses the caller-supplied `payloadSize` from the proof header when it checks `chunkLen`:

```
// CelestiaDAProofValidator.sol::validateReadPreimage
ReadProofHeader memory header = _decodeReadProofHeader(customProof);
require(header.offset == offset, "Offset mismatch");
require(header.chunkLen == _expectedChunkLen(offset, header.payloadSize),
    "Invalid chunkLen");
```

But later, `_extractChunk()` only checks that `payloadSize` matches the share sequence length when the read starts in the first share:

```
// CelestiaDAProofValidator.sol::_extractChunk
uint256 firstRel = _payloadOffsetToShareRel(offset);

bytes memory firstShare = sharesProof.data[0];
uint256 firstSharePayloadOffset = _payloadDataStartInShare(firstRel);
require(firstShare.length >= firstSharePayloadOffset, "Invalid share header");

if (firstRel == 0) {
    require(payloadSize == _decodeSequenceLen(firstShare), "Payload size
    mismatch");
}
```

If the read starts in a later share, `payloadSize` is not checked against the real sequence length. That means `payloadSize` is only a caller-controlled header value.

Because of this, an attacker can set `payloadSize` bigger than the real payload size and make the contract return bytes past the real end of the payload.

Impact

A malicious prover can make `validateReadPreimage()` return bytes past the real end of the payload.

This means the one-step proof path may read an incorrect 32-byte chunk for the challenged certificate. That can break replay and may lead to an incorrect dispute result.

Recommendations

Always check `payloadSize` against the real sequence length from the first share.

For continuation share reads, the proof should also include a proof for the first share so the contract can check the real payload size before returning bytes.

Remediation

This issue has been acknowledged by Celestia Labs, and fixes were implemented in the following commits:

- `5c4d1cfa` ↗
- `f6d3e837` ↗

3.2. Read preimage proof does not bind proved shares to the share picked by the certificate

Target	CelestiaDAProofValidator.sol		
Category	Soundness	**Severity**	High
Likelihood	Medium	**Impact**	High

Description

The `validateReadPreimage()` flow checks two things separately:

1. The caller-supplied header matches the certificate and the requested `offset`.
2. The supplied `SharesProof` is valid under the attested `(blockHeight, dataRoot)`.

What it does not check is that the proved shares are the exact shares picked by the certificate.

The function first checks the header against the requested `offset` and the certificate share range:

```
// CelestiaDAProofValidator.sol::validateReadPreimage
ReadProofHeader memory header = _decodeReadProofHeader(customProof);
require(header.offset == offset, "Offset mismatch");
require(header.chunkLen == _expectedChunkLen(offset, header.payloadSize),
    "Invalid chunkLen");

CertificateData memory certData = _extractCertificateData(certificate);
_validateShareRange(certData, offset, header.firstShareIndexInBlob,
    header.shareCount);
```

However, `header.firstShareIndexInBlob` and `header.shareCount` are fully caller-controlled; `_validateShareRange()` only checks that these header values match the share window that should be read for the given `offset` — it does not check that the decoded `SharesProof` really proves that same window.

The function then verifies the Celestia proof:

```
// CelestiaDAProofValidator.sol::validateReadPreimage
(address encodedBlobstream, SharesProof memory sharesProof) =
    abi.decode(sharesProofData, (address, SharesProof));
require(encodedBlobstream == blobstreamX, "Blobstream address mismatch");

require(
```

```
    sharesProof.attestationProof.tuple.height == certData.blockHeight,
    "Shares proof height does not match certificate blockHeight"
);
require(
    sharesProof.attestationProof.tuple.dataRoot == certData.dataRoot,
    "Shares proof dataRoot does not match certificate dataRoot"
);

(bool valid,)
    = DAVerifier.verifySharesToDataRootTupleRoot(IDAOracle(blobstreamX),
    sharesProof);
require(valid, "Invalid Celestia shares inclusion proof");
```

However, DAVerifier only checks that the supplied shares are included in the proved rows and that those rows are included in the attested `dataRoot`. It counts shares using the proof-supplied `beginKey` and `endKey` values and checks those ranges against the proof-supplied row roots:

```
// DAVerifier.sol::verifySharesToDataRootTupleRootProof
uint256 numberOfSharesInProofs = 0;
for (uint256 i = 0; i < _shareProofs.length; i++) {
    numberOfSharesInProofs += _shareProofs[i].endKey
    - _shareProofs[i].beginKey;
}

// ...
if (
    !NamespaceMerkleTree.verifyMulti(
        _rowRoots[i],
        _shareProofs[i],
        _namespace,
        s
    )
) {
    return (false, ErrorCodes.InvalidSharesToRowsProof);
}
```

There is no check that compares the share position proved by `SharesProof` with `header.firstShareIndexInBlob`. Because of this, an attacker can set the header to the expected share window while giving a valid `SharesProof` for a different share range from the same Blobstream attested `dataRoot`.

Impact

A malicious prover can make `validateReadPreimage()` return real bytes from the right Celestia block but from the wrong share.

Recommendations

Add a check that compares the share position proved by `shareProofs` and `rowProofs` with the expected `firstShareIndexInBlob` and `shareCount`.

In other words, the contract should check the proved row, the proved start inside that row, and the total number of proved shares. Any proof for a different share range should be rejected, even if it is valid under the same `dataRoot`.

Remediation

This issue has been acknowledged by Celestia Labs, and fixes were implemented in the following commits:

- 41269136 ↗
- ba32363c ↗

3.3. Validating the certificate depends on `BlobstreamX.verifyAttestation` terminating

Target	CelestiaDAProofValidator.sol, BlobstreamX		
Category	Integration Risks	**Severity**	High
Likelihood	Medium	**Impact**	High

Description

The function `validateCertificate` invokes the external Blobstream contract and returns its result directly:

```
return IDAOracle(blobstreamX).verifyAttestation(
    attestationProof.tupleRootNonce, attestationProof.tuple,
    attestationProof.proof
);
```

This implies that the `validateCertificate` function depends on the `BlobstreamX::verifyAttestation` function not reverting and terminating properly.

Impact

If the `BlobstreamX::verifyAttestation` function could revert or not terminate properly (e.g., fall into an infinite loop and exhaust gas) for some input, it will violate the requirement that the `validateCertificate` function must not revert. An attacker could provide the input that makes the `BlobstreamX::verifyAttestation` function revert to the dispute game in order to trigger chain halt.

Though the scope of this audit does not include the external dependencies of provided codebase, we have informally reviewed the behavior of the `BlobstreamX::verifyAttestation` function of the latest BlobstreamX contract. We discovered that this function will fall into an unbounded loop when `_proof.numLeaves` is greater than 2^{255} because the `getStartingBit` function starts the loop under the condition `(1 << startingBit) < numLeaves`, which will be true regardless of increments of `startingBit`.

Recommendations

Consider ensuring that `BlobstreamX::verifyAttestation` terminates properly and does not revert. This could include using a BlobstreamX instance whose `verifyAttestation` function is comprehensively reviewed in this aspect.

Please note that our informal review on the `BlobstreamX::verifyAttestation` function does not guarantee that there is no other possible way to make this function revert or fall into an unbounded loop.

Remediation

This issue has been acknowledged by Celestia Labs, and a fix was implemented upstream in Blobstream in commit b8482c59 ↗.

3.4. The function `validateCertificate` may return false for a valid proof under some gas limits

Target	CelestiaDAProofValidator.sol		
Category	Soundness	**Severity**	Low
Likelihood	Low	**Impact**	Low

Description

The function `validateCertificate()` decodes `AttestationProof` with an external call wrapped in `try catch`:

```
AttestationProof memory attestationProof;
try this.decodeAttestationProof(attestationProofData)
    returns (AttestationProof memory decoded) {
    attestationProof = decoded;
} catch {
    return false;
}
```

The goal is to return `false` for bad ABI data instead of reverting. But this also makes the result depend on the gas given to the inner call.

An EVM call forwards at most 63/64 of the remaining gas and keeps 1/64 in the caller. Because of that, it may be possible for `decodeAttestationProof()` to run out of gas while the outer `catch` block still has enough gas to execute `return false`.

If that happens, `validateCertificate()` would return `false` even for a valid certificate proof. In other words, this creates a possible false negative path where the result depends on gas, not only on proof validity.

Impact

A valid certificate may look invalid if the inner decode call runs out of gas. We think this is unlikely and may be hard to trigger in the full Arbitrum call path, so the practical impact may be limited.

Still, if it does happen, the on-chain challenge flow may reject a valid certificate for gas reasons instead of proof reasons.

Recommendations

Do not use an external self-call with `try catch` for this decode step.

Use an internal decode path that returns `false` for bad data without depending on call gas.

Remediation

This issue has been acknowledged by Celestia Labs, and fixes were implemented in the following commits:

- 46a0f09e
- 047b13d5

3.5. Unsafe `uint64` to `big.Int` conversion

Target	proof.go, celestia.go		
Category	Coding Mistakes	**Severity**	Low
Likelihood	Low	**Impact**	Low

Description

In multiple locations, `uint64` values are cast to `int64` before being passed to `big.NewInt()`. Since `int64` has a maximum value of $2^{63} - 1$, any `uint64` value exceeding that range will silently overflow, producing an incorrect (negative) `big.Int`.

```
func toAttestationProof(
    nonce uint64,
    height uint64,
    blockDataRoot [32]byte,
    dataRootInclusionProof *blobstream.DataRootTupleInclusionProof,
) AttestationProof {
[...]
    return AttestationProof{
        TupleRootNonce: big.NewInt(int64(nonce)),
        Tuple: DataRootTuple{
            Height:   big.NewInt(int64(height)),
            DataRoot: blockDataRoot,
        },
        Proof: BinaryMerkleProof{
            SideNodes: sideNodes,
            Key:       big.NewInt((*dataRootInclusionProof).Index),
            NumLeaves: big.NewInt((*dataRootInclusionProof).Total),
        },
    }
}
```

```
func (c *CelestiaDA) GetProof(ctx context.Context, msg []byte) ([]byte, error)
    {
[...]
    tuple := blobstreamx.DataRootTuple{
        Height:   big.NewInt(int64(blobPointer.BlockHeight)),
        DataRoot: [32]byte(header.DataHash),
    }
[...]
```

```go
func (c *CelestiaDA) resolveBlobstreamEvent(
[...]
    latestCelestiaBlock, err := blobstream.LatestBlock(&bind.CallOpts{
        Pending:     false,
        BlockNumber: big.NewInt(int64(latestBlockNumber)),
        Context:     ctx,
    })

[...]
func (c *CelestiaDA) generateCelestiaProof(
[...]
    tuple := blobstreamx.DataRootTuple{
        Height:   big.NewInt(int64(certificate.BlockHeight)),
        DataRoot: [32]byte(header.DataHash),
    }
```

Impact

While overflow may be unlikely in practice, the code provides no protection against it.

Recommendations

We suggest using `new(big.Int).SetUint64()`, which handles the full `uint64` range correctly; see the Go documentation ↗ (`Int.SetUint64`).

Remediation

This issue has been acknowledged by Celestia Labs, and a fix was implemented in commit `138366e3` ↗.

3.6. Missing retry logic on postsubmission read operations

Target	celestia.go		
Category	Coding Mistakes	**Severity**	Low
Likelihood	Low	**Impact**	Low

Description

In the method `(c *CelestiaDA) Store`, the blob submission to Celestia is retried with the gas price bumping on failure. However, the two subsequent read operations from Celestia are called without any retry logic.

```
func (c *CelestiaDA) Store(ctx context.Context, message []byte) ([]byte,
    error) {
[...]
    gasPrice := -1.0
    for !submitted {
        // add submit options
        submitOptions := &blob.SubmitOptions{}
        state.WithGasPrice(gasPrice)(submitOptions)
        if c.Cfg.ExperimentalTxClient {
            height, err = c.TxClient.Blob.Submit(ctx, []*blob.Blob{dataBlob},
    submitOptions)

[...]
        if err != nil {
            switch {
            case strings.Contains(err.Error(), ErrTxTimedout.Error()),
    strings.Contains(err.Error(), ErrTxAlreadyInMempool.Error()),
    strings.Contains(err.Error(), ErrTxIncorrectAccountSequence.Error()):
                log.Warn("Failed to submit blob, bumping gas price and
    retrying...", "err", err)
                if gasPrice == -1.0 {
                    gasPrice = c.Cfg.GasPrice
                } else {
                    gasPrice = gasPrice * c.Cfg.GasMultiplier
                }

                celestiaGasRetries.Inc(1)
                continue
[...]
    }
```

```
[...]
    // we fetch the blob so that we can get the correct start index in the
    square
    dataBlob, err = c.ReadClient.Blob.Get(ctx, height, *c.Namespace,
    dataBlob.Commitment)
    if err != nil {
        log.Warn("could not fetch blob", "err", err)
        celestiaFailureCounter.Inc(1)
        return nil, err
    }
[...]
    header, err := c.ReadClient.Header.GetByHeight(ctx, height)
    if err != nil {
        celestiaFailureCounter.Inc(1)
        log.Warn("Header retrieval error", "err", err)
        return nil, err
    }
```

This is inconsistent with the `(c *CelestiaDA) Read` method, where every Celestia `Get` operation is wrapped in a `retryWithBackoff` helper function.

Impact

A transient error on either `Blob.Get` or `Header.GetByHeight` will cause the entire `Store` to return an error despite the blob having been submitted successfully. Since every successful `Blob.Submit` operation on Celestia requires transaction fees, situations that can result in double-spending should be minimized.

Recommendations

Implement retry logic on both `Blob.Get` and `Header.GetByHeight` calls in the `Store` method.

Remediation

This issue has been acknowledged by Celestia Labs, and a fix was implemented in commit `61f6535e` ↗.

3.7. Possible infinite loop in backwards search in `filter()`

Target	celestia.go		
Category	Coding Mistakes	**Severity**	Low
Likelihood	Low	**Impact**	Low

Description

If no matching event exists in Blobstream and `backwards = true`, the `filter` function loop slides the search window back in 5,000-block increments until `start` reaches zero. Once there, the loop continues indefinitely, rescanning the same range of blocks (blocks 0–1,000) until the context is canceled. There is no explicit check that detects that the full block range has been exhausted.

Impact

This loop condition can create an infinite loop that will create unnecessary load for the L1 RPC endpoint.

Recommendations

Add an explicit termination condition in the backwards branch when the window reaches the bottom.

Please note that according to the Arbitrum Nitro DA integration guide ↗, there is a critical rule for the function `daprovider_generateCertificateValidityProof`. Invalid certificates, including those with bad format, bad signatures, or untrusted signers, must return a *successful response* with `claimedValid=0` in the proof, not an error.

Errors should only be returned for genuine transient issues, such as network failures (e.g., unreachable L1, database down), RPC time-outs, or similar infrastructure problems. The function `generateCertificateAttestationProof` will catch the errors of type `CertificateValidationError` and return `nil`, `nil` instead to handle this.

Remediation

This issue has been acknowledged by Celestia Labs, and a fix was implemented in commit 989e0357 ↗.

3.8. Noncompliant error messages for invalid certificate errors in reader methods

Target	types/reader.go		
Category	Coding Mistakes	**Severity**	Low
Likelihood	Low	**Impact**	Low

Description

The Arbitrum Nitro DA integration guide ↗ specifies that `RecoverPayload` (and all reader methods) must return errors containing the substring `"certificate validation failed"` whenever a certificate is invalid — including cases of bad format.

However, parsing errors surfaced by `cert.ExtractFromSequencerMessage` and its callees (`ParseCelestiaCertificate`, `CelestiaDACertV1.UnmarshalBinary`) are returned unwrapped directly to the caller. Errors such as `ErrInvalidCertificateLength`, `ErrInvalidCertificateHeader`, and related do not carry the required `"certificate validation failed"` prefix. Errors raised later in `celestiaReader.Read` — for example, `certificateValidationError("data root mismatch")` — are correctly wrapped.

Impact

Nitro behavior changes depending on the error condition. In particular, in the case of an invalid certificate, the batch is treated as empty and the syncing is allowed to continue. In the case of other errors, the syncing stops immediately. The two different behaviors should distinguish between errors that are transient and can be solved by retrying later, such as infrastructure failures, and errors that cannot be solved.

Returning a different error when a `CertificateValidationError` should be returned can cause the Nitro node to never sync successfully.

Recommendations

Wrap all errors returned from `cert.ExtractFromSequencerMessage` in `RecoverPayloadFromCelestiaBatch` using `certificateValidationError(...)` or an equivalent wrapper that injects the required `"certificate validation failed"` substring before propagating.

Remediation

This issue has been acknowledged by Celestia Labs, and fixes were implemented in the following commits:

- 6cf6521a
- 232cfd0b

3.9. Noncompliant error in the case of an empty batch

Target	types/reader.go		
Category	Coding Mistakes	**Severity**	Low
Likelihood	Low	**Impact**	Low

Description

The Arbitrum Nitro DA integration guide ↗ specifies that `RecoverPayload` (and all reader methods) should handle an empty batch as a valid response. The function `RecoverPayloadFromCelestiaBatch`, which is called for every read operation, returns an error when an empty message is retrieved from Celestia:

```
func RecoverPayloadFromCelestiaBatch(
[...]
    if len(result.Message) == 0 {
        return nil, nil, errors.New("empty payload returned from Celestia")
    }
```

Impact

Nitro behavior for error conditions is to stop processing. Since the empty batch is not a transient error or an infrastructure error that can be solved and later the processing can be resumed, returning an error in this case can prevent the Nitro node from ever syncing correctly.

Recommendations

Remove the early return with the error message and return a successful response with `Payload": null` or `"Payload": "0x"` as recommended by the integration guide.

Remediation

This issue has been acknowledged by Celestia Labs, and a fix was implemented in commit `653b0fe3` ↗.

3.10. Insufficient test coverage

Target	Project-wide		
Category	Protocol Risks	**Severity**	Medium
Likelihood	N/A	**Impact**	Medium

Description

While the codebase features meaningful tests for several security-sensitive components, some portions of the codebase remain untested — most notably, the entire `daserver/types/tree/` package (~407 lines). In addition, more tests checking for compliance with the integration guide ↗ would be advisable, especially around edge cases in error handling.

Impact

Good test coverage has multiple effects:

- It finds bugs and design flaws early (preaudit or prerelease).
- It gives insight into areas for optimization (e.g., gas cost).
- It displays code maturity.
- It bolsters customer trust in the product.
- It improves understanding of how the code functions, integrates, and operates — for developers and auditors alike.
- It increases development velocity long-term.

The last point may seem contradictory given the time investment to create and maintain tests. However, tests help developers trust their own changes. It is difficult to know if a code refactor — or even just a small one-line fix — breaks other code if there are no tests. This is especially true for new developers or those returning to the code after a prolonged absence.

Tests have your back here. They are an excellent indicator that the existing functionality was most likely not broken by a change to the code. Without a comprehensive test suite, the above benefits are lost. This increases the likelihood of bugs and vulnerabilities going unnoticed until after deployment, which can be costly and damaging to the project's reputation.

Recommendations

We recommend expanding the test suite to ensure extensive and thorough test coverage, specifically with the following.

- **Tree package tests.** Add unit tests for daserver/types/tree/.

- **Integration-guide–compliance tests.** Add tests ensuring that edge cases are handled according to the integration guide ↗.

Remediation

This issue has been acknowledged by Celestia Labs, and fixes were implemented in the following commits:

- `f153b2ae` ↗
- `653b0fe3` ↗
- `6cf6521a` ↗
- `61f6535e` ↗
- `232cfd0b` ↗
- `c7b20a28` ↗

4. Discussion

The purpose of this section is to document miscellaneous observations that we made during the assessment. These discussion notes are not necessarily security related and do not convey that we are suggesting a code change.

4.1. Potential memory-corruption risk in prefix slice reuse

The existing TODO comment in tree/hash.go suggests the intention to give `leafPrefix` and `innerPrefix` a larger predefined capacity. While the code is safe at the time of writing, `append` reallocates when there is no spare capacity on the backing array, and this automatic reallocation assumption would silently break with larger predefined capacity.

With larger preallocated capacity, subsequent `append` calls to the same `leafPrefix` slice would share the same backing array, since no reallocation would be performed. This means a later append would overwrite the bytes written by an earlier one, silently corrupting the `preimage` of a previous call.

```go
// TODO: make these have a large predefined capacity
var (
    leafPrefix  = []byte{0}
    innerPrefix = []byte{1}
)
[...]
// returns tmhash(0x00 || leaf)
func leafHash(record func(bytes32, []byte, arbutil.PreimageType), leaf []byte)
    []byte {
    preimage := append(leafPrefix, leaf...)
[...]
```

As a best practice, the append should always force a fresh allocation, making it robust regardless of the backing capacity:

```go
preimage := append(append([]byte{}, leafPrefix...), leaf...)
```

This is purely a defensive suggestion for future-proofing. No current behavior is affected.

Additionally, in the same file, `innerHash` can return the already computed hash variable directly instead of recomputing it.

4.2. Unused `timeout` parameter in `writerForCelestia.Store` RPC method

The `timeout` parameter in the `Store` RPC method in the writer.go file is accepted but not used. Per the Arbitrum Nitro DA API integration guide ↗, this parameter is intended to control data expiry (blob lifetime) rather than request duration. The parameter is kept to satisfy the interface contract; however, the Celestia Labs team confirmed that they do not currently support blob lifetimes, making the parameter unused until that functionality is available.

```
func (c *writerForCelestia) Store(
    message []byte,
    timeout uint64,
) containers.PromiseInterface[[]byte] {
    return containers.DoPromise(context.Background(), func(ctx
    context.Context) ([]byte, error) {
        cert, err := c.celestiaWriter.Store(ctx, message)
        if err != nil {
            log.Error("Returning error from Celestia writer", "err", err)
            return nil, err
        }
        return cert, nil
    })
}
```

4.3. Silent validation failure in `GetProof`

When `blobstream.VerifyAttestation` returns `valid = false`, the function falls through to the final return statement:

```
func (c *CelestiaDA) GetProof(ctx context.Context, msg []byte) ([]byte, error)
    {
[...]
    valid, err := blobstream.VerifyAttestation(
        &bind.CallOpts{},
        event.ProofNonce,
        tuple,
        proof,
    )
    if err != nil {
        log.Warn("could not verify attestation", "err", err)
        celestiaValidationFailureCounter.Inc(1)
        return nil, err
    }
```

```
    log.Info("Verified Celestia Attestation", "height",
    blobPointer.BlockHeight, "valid", valid)

    if valid {
[...]
    return proofData, nil
    }

    celestiaValidationFailureCounter.Inc(1)
    return nil, err
```

Since no error path sets `err` at this point, this returns a silent validation failure (`nil, nil`) to the caller.

As this is a legacy interface, we raise this here for discussion. We leave it to Celestia Labs to assess the impact and decide whether a fix is necessary.

4.4. Iterator error handling causes silent discard of a valid event in `filter`

In the `filter` function in celestia.go, in the inner loop, if a matching event has been found but `eventsIterator.Error()` or `eventsIterator.Close()` returns an error, the function returns `nil, err`, discarding the valid event. The iterator should be closed before returning, and a found event should take precedence over a noncritical close error.

```
func (c *CelestiaDA) filter(
[...]
    for {
[...]
        var event *blobstreamx.BindingsDataCommitmentStored
        for eventsIterator.Next() {
            e := eventsIterator.Event
            if e.StartBlock <= celestiaHeight && celestiaHeight < e.EndBlock {
                event = &blobstreamx.BindingsDataCommitmentStored{
                    ProofNonce:     e.ProofNonce,
                    StartBlock:     e.StartBlock,
                    EndBlock:       e.EndBlock,
                    DataCommitment: e.DataCommitment,
                }
                break
            }
        }
        if err := eventsIterator.Error(); err != nil {
```

```
            return nil, err
        }
        err = eventsIterator.Close()
        if err != nil {
            return nil, err
        }
        if event != nil {
            log.Info("Found Data Root submission event", "proof_nonce",
    event.ProofNonce, "start", event.StartBlock, "end", event.EndBlock)
            return event, nil
        }
[...]
```

5. System Design

This provides a description of the high-level components of the system and how they interact, including details like a function's externally controllable inputs and how an attacker could leverage each input to cause harm or which invariants or constraints of the system are critical and must always be upheld.

Not all components in the audit scope may have been modeled. The absence of a component in this section does not necessarily suggest that it is safe.

5.1. Component: DA server

Description

The `nitro-das-celestia` project is a DA server sidecar for the Arbitrum Nitro stack used to post and retrieve batch data via Celestia as a data-availability layer. This server allows to use Celestia for data availability instead of Arbitrum AnyTrust. It is designed to run locally alongside the Nitro node and is never exposed to the internet. The Nitro node fully trusts all responses from this server, and the server in turn fully trusts the configured Celestia nodes.

The server implements the DA server specification described in the Arbitrum DA API integration guide ↗, which defines how a custom DA backend plugs into Nitro via the `daprovider` interface.

Project structure

The core logic of the DA server is implemented in the daserver/ subdirectory, and cmd/celestiadaserver.go provides the entry point and CLI argument handling.

The files inside the daserver/ subdirectory provide the following functionalities:

- celestia.go — core Celestia write/read/proof logic
- rpc_server.go — RPC server instantiation and wiring
- proof.go — proof format
- cert/celestia_cert.go — CelestiaDACertV1 binary format
- types/ and validator/ — reader, writer, and validator adapters, types, and logic

RPC interface

The server exposes a single HTTP JSON-RPC endpoint with two namespaces, registered on the same listener:

1. The **`celestia` namespace** — legacy interface, used by the legacy Celestia Nitro fork
    - `celestia_store(message) → serializedCert` — submits a blob to Celestia and returns a serialized CelestiaDACertV1

- `celestia_read(cert) → ReadResult` — retrieves blob data and inclusion proof material
- `celestia_getProof(blobPointerBytes) → proofData` — constructs a BlobstreamX proof (legacy path)

2. The **`daprovider` namespace** — new official Arbitrum DA provider API as specified in the integration guide, which implements the three interfaces from Nitro's daprovider requirements over RPC
    - **Reader** (`getSupportedHeaderBytes`, `recoverPayload`, `collectPreimages`, `recoverPayloadAndPreimages`) — payload retrieval from a sequencer message containing an embedded cert
    - **Writer** (`getMaxMessageSize`, `store`, and chunked streaming via `startChunkedStore` / `sendChunk` / `commitChunkedStore`) — blob submission to Celestia returning a serialized cert
    - **Validator** (`generateReadPreimageProof`, `generateCertificateValidityProof`) — BlobstreamX-backed fraud proof generation for individual 32-byte preimage chunks and certificate proof of validity

Error-handling rules in the integration guide

The integration document mentions that the DA provider is a trusted component. The implementer must ensure that responses in case of errors are returned according to the expectations of Nitro. Transient errors such as temporary network issues should return an error, while nontransient issues like an invalid certificate should return a `CertificateValidationError` for the reader RPC methods.

For transient errors, Nitro will retry the failed operation, while for nontransient issues, Nitro will give up and treat the batch as empty. Returning an unspecified error in the case of an invalid certificate would stop the syncing of the rollup without the possibility to recover (see Finding 3.8. ↗). A batch with zero length should be considered as a valid response in order for the syncing to continue (see Finding 3.9. ↗).

Similar considerations apply for the validator RPC methods. The `generateCertificateValidityProof` method follows a slightly different rule for error handling: in the case of invalid certificates (same conditions that would return `CertificateValidationError` in reader methods), it must return a successful response with the `claimedValid` field equal to zero. The guide notes that it is critical to not return an error in those specific circumstances.

Invariants

- A successful `store` operation must return a valid certificate pointing to a confirmed blob on Celestia. The `store` method features automatic retry with gas price adjustment for network congestion.
- Successful `submit` operations on Celestia, called by the `store` method, should result in a

successful response to avoid that the Nitro node resubmits the same data (avoid double-spending transaction fees; see Finding 3.6. ↗).
- Reader methods should validate the input certificate, return appropriate error messages in case of errors, and provide the corresponding data present on Celestia (see Findings 3.8. ↗ and 3.9. ↗).
- Validator methods interact with BlobstreamX to verify the attestation of proofs returned and to assess the validity of the provided certificate.

Test coverage

Cases covered

- Certificate marshaling and unmarshaling, including error cases
- Reader logic
- Writer logic (success paths only)
- Proof-packing logic
- AWS KMS create, import, and configuration errors
- Validator logic
- Read data validation logic
- Integration tests for store/read flows (with error handling) and `generateCertificateValidityProof`

Cases not covered

- The `daprovider` read paths where malformed or invalid certificates return a `CertificateValidationError`
- The `daprovider` read paths where an empty batch still returns success
- RPC server unit tests
- The `daprovider` chunked store flow
- The `types/tree` unit tests

Integration tests in this repo test the DA server against live Celestia infrastructure; Blobstream-related paths additionally need an Ethereum JSON-RPC endpoint and Blobstream contract address and are skipped if not configured.

Attack surface

The DA server is treated as trusted internal infrastructure; it defaults to binding to `localhost` and should only be reachable by the Nitro host or a tightly segmented network. This implementation does not authenticate callers on its JSON-RPC API, so exposure lets an attacker spend fees via Celestia submissions using the server's key and feed arbitrary DA responses to Nitro, which invalidates the usual Celestia DA assumption for that node.

5.2. Component: DA proof validator smart contract

Description

CelestiaDAProofValidator is the smart contract that validates Celestia DA certificates and preimage proofs. It implements DA API that Arbitrum Nitro stack employs — one may configure their Arbitrum Nitro chain to use Celestia as a data-availability provider through this contract.

This contract will be deployed on the parent chain of the Arbitrum Nitro chain and interact with challenge protocol contracts in order to resolve disputes. The methods provided by this contract, `validateCertificate` and `validateReadPreimage`, are largely stateless; the caller `OneStepProverHostIo`, which is one of the challenge protocol contracts, is responsible to utilize these functions to track the status of disputes and properly resolve them.

Invariants

- The `validateCertificate` function must return true if the certificate is valid — and false otherwise.
- The `validateCertificate` function must terminate normally (returning true or false) and not revert, given that the gas is enough and the input calldata is valid ABI encoding of parameters.
- The `validateReadPreimage` function must revert when the certificate in the proof does not match the provided certificate hash.
- The `validateReadPreimage` function must return the requested preimage data if the given proof is valid.

Test coverage

Cases covered

- The `validateCertificate` function returns true when a valid certificate is provided.
- The `validateCertificate` function returns false when a certificate with an invalid length field is provided.
- The `validateCertificate` function returns false when a certificate with an incorrect version is provided.
- The `validateCertificate` function returns false when a proof is not decodable.
- The `validateCertificate` function returns false when a tuple (height and data root) of the proof does not match the one in the certificate.
- The `validateCertificate` function returns false when attestation could not be verified through BlobstreamX.
- The `validateReadPreimage` function returns the preimage data of the requested offset when a valid proof is provided.
- The `validateReadPreimage` function reverts when a certificate does not match the certificate hash in the parameter.

- The `validateReadPreimage` function reverts when a proof with an invalid length field is provided.
- The `validateReadPreimage` function reverts when a proof with an incorrect version is provided.
- The `validateReadPreimage` function reverts when a proof is from a different BlobstreamX instance.
- The `validateReadPreimage` function reverts when an invalid share proof is provided.
- The `validateReadPreimage` function reverts when invalid data is provided with a valid certificate.
- The `validateReadPreimage` function reverts when invalid data is provided with an invalid certificate.
- The `validateReadPreimage` function reverts when an offset in the proof does not match the one requested.
- The `validateReadPreimage` function reverts when chunk length in the proof does not match the one expected.
- The `validateReadPreimage` function reverts when the first share index does not match the one expected with offset.
- The `validateReadPreimage` function reverts when the number of shares does not match the one expected with offset.
- The `validateReadPreimage` function reverts when the payload size of each share does not match the one expected with offset.
- The `validateReadPreimage` function reverts when a tuple (height and data root) of the proof does not match the one in the certificate.
- The `validateReadPreimage` function reverts when a certificate does not match the certificate hash in the parameter.

Cases not covered

- The `validateCertificate` function returns false when a certificate or proof is too short to parse correctly.
- The `validateReadPreimage` function reverts when a certificate or proof is too short to parse correctly.
- The `validateReadPreimage` function reverts when indexes of the provided shares do not match the ones provided in the header.

Attack surface

An actor on the dispute game may submit an arbitrary certificate and proof to the protocol. A dishonest actor on the dispute game may try to submit a rogue certificate and proof the contract deems to be valid although it should not and winning the game when they should not. Also, an attacker may try to submit a certificate and proof that makes the `validateCertificate` function revert, which will result in a chain halt. Since the certificate and proof could be arbitrarily provided by a malicious party, it is important to parse the certificate and proof defensively (i.e., try to cover every edge case).

Notably, developers should keep in mind that the `validateCertificate` function must terminate normally and not revert. Extra caution must be exercised on every aspect of `validateCertificate`, including but not limited to parsing, data processing, consistency over multiple inputs, and so on. External dependencies the protocol use during the `validateCertificate` (e.g., BlobstreamX) are not the exception to this rule. Since those dependencies were outside the scope of this audit, developers must ensure that this assumption holds. Simply catching the revert might not be enough to handle all edge cases — for example, the issue of BlobstreamX described in Finding 3.3. ↗ would not be properly handled even if the contract implemented the logic to catch errors from BlobstreamX.

Most of the contract is stateless, but the owner role is implemented in the contract. The owner may change the `maxMessageSize`, which does not affect on-chain behaviors.

6. Assessment Results

During our assessment on the scoped nitro-das-celestia contracts, we discovered 10 findings. No critical issues were found. Three findings were of high impact, one was of medium impact, and six were of low impact.

6.1. Disclaimer

This assessment does not provide any warranties about finding all possible issues within its scope; in other words, the evaluation results do not guarantee the absence of any subsequent issues. Zellic, of course, also cannot make guarantees about any code added to the project after the version reviewed during our assessment. Furthermore, because a single assessment can never be considered comprehensive, we always recommend multiple independent assessments paired with a bug bounty program.

For each finding, Zellic provides a recommended solution. All code samples in these recommendations are intended to convey how an issue may be resolved (i.e., the idea), but they may not be tested or functional code. These recommendations are not exhaustive, and we encourage our partners to consider them as a starting point for further discussion. We are happy to provide additional guidance and advice as needed.

Finally, the contents of this assessment report are for informational purposes only; do not construe any information in this report as legal, tax, investment, or financial advice. Nothing contained in this report constitutes a solicitation or endorsement of a project by Zellic.